

Documentation Technique

- **Docker** est l'élément principal de l'architecture, le fait qu'il isole chaque service dans un conteneurs assure une installation facile et identique sur tous les environnements.

Les services Docker :

Serveur Nginx 1.26-alpine

Le projet utilise un serveur web Nginx chargé de gérer les requêtes HTTP et de les transmettre au conteneur PHP-FPM. Nginx a été retenu pour sa rapidité, sa stabilité et sa large utilisation dans les environnements modernes.

PHP 8.3-fpm-alpine

Les versions alpine sont des versions plus légère et récente. Malgré une architecture plus minimaliste (moins d'outils système), elle reste une option parfaite pour un petit projet Symfony.

MySQL 8.0

MySQL est utilisée comme base de données principale de l'application. Elle a été administrer via **MySQL WorkBench** durant le développement du projet.

MongoDB 7.0

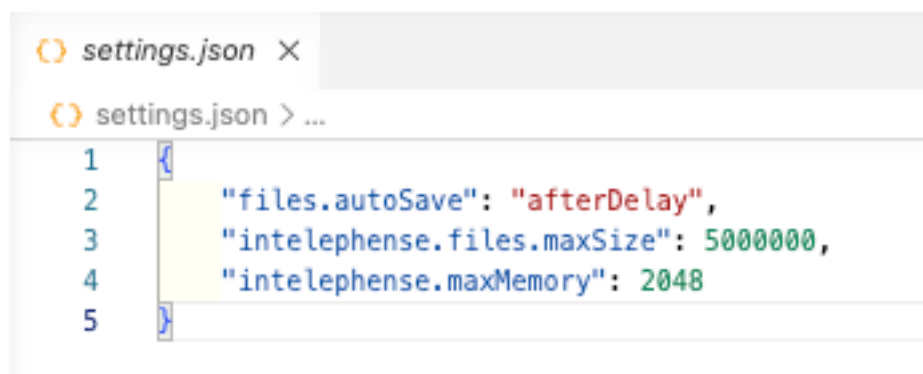
Base de données non relationnel qui permet de stocker les statistiques de l'application sous forme de document. Le modèle orienté document permet une structure plus souple pour des données statistiques ne nécessitant pas un schéma relationnel complexe.

- **Le framework Symfony 7.4 (LTS)** facilite l'organisation du code grâce à une architecture **MVC claire**, un système de **routage intégré**, une **gestion simplifiée** des formulaires ainsi que des **mécanismes de sécurité** intégrés pour l'authentification, **les rôles** ou la réinitialisation des mots de passe. Le projet utilise une version **LTS** (Long Term Support) afin de privilégier la stabilité pendant ou après le développement.

- **Git et GitHub** assure la gestion des versions (le versioning) du code, et le partage, la sauvegarde du projet via GitHub. Les commits utilisés à chaque sauvegarde d'une version est au format Angular en anglais.
- **Visual Studio Code (VS Code)** a été utilisé comme environnement de travail et de développement durant la réalisation du projet. Des extensions comme GitLense on été utile pour visualiser les versions et branches crée directement depuis VS Code.

- **Configuration de l'environnement de travail**

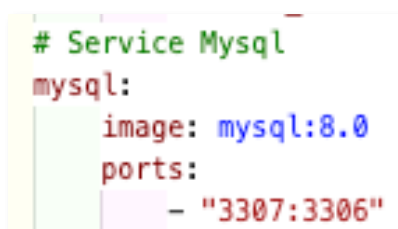
Sur Visual Studio Code, certaines extensions comme PHP intelephense ont été utilisés et configurés :



```
settings.json X
settings.json > ...
1 {
2   "files.autoSave": "afterDelay",
3   "intelephense.files.maxSize": 5000000,
4   "intelephense.maxMemory": 2048
5 }
```

Le fichier **settings.json** permet de configurer **le comportement de VS Code** et de ses extensions éventuelles. Les paramètres ci-dessus permettent à PHP Intelephense d'éviter des ralentissements ou **blocages lors de l'indexation** et de l'analyse des fichiers du projet Symfony.

Pour utiliser **MySQL Workbench** qui ne fait pas partie d'un service Docker, le conteneur MySQL doit pouvoir avoir un accès en dehors de celui-ci, pour cela, on utilise **ports**: dans docker compose, indiquant les ports d'accès :

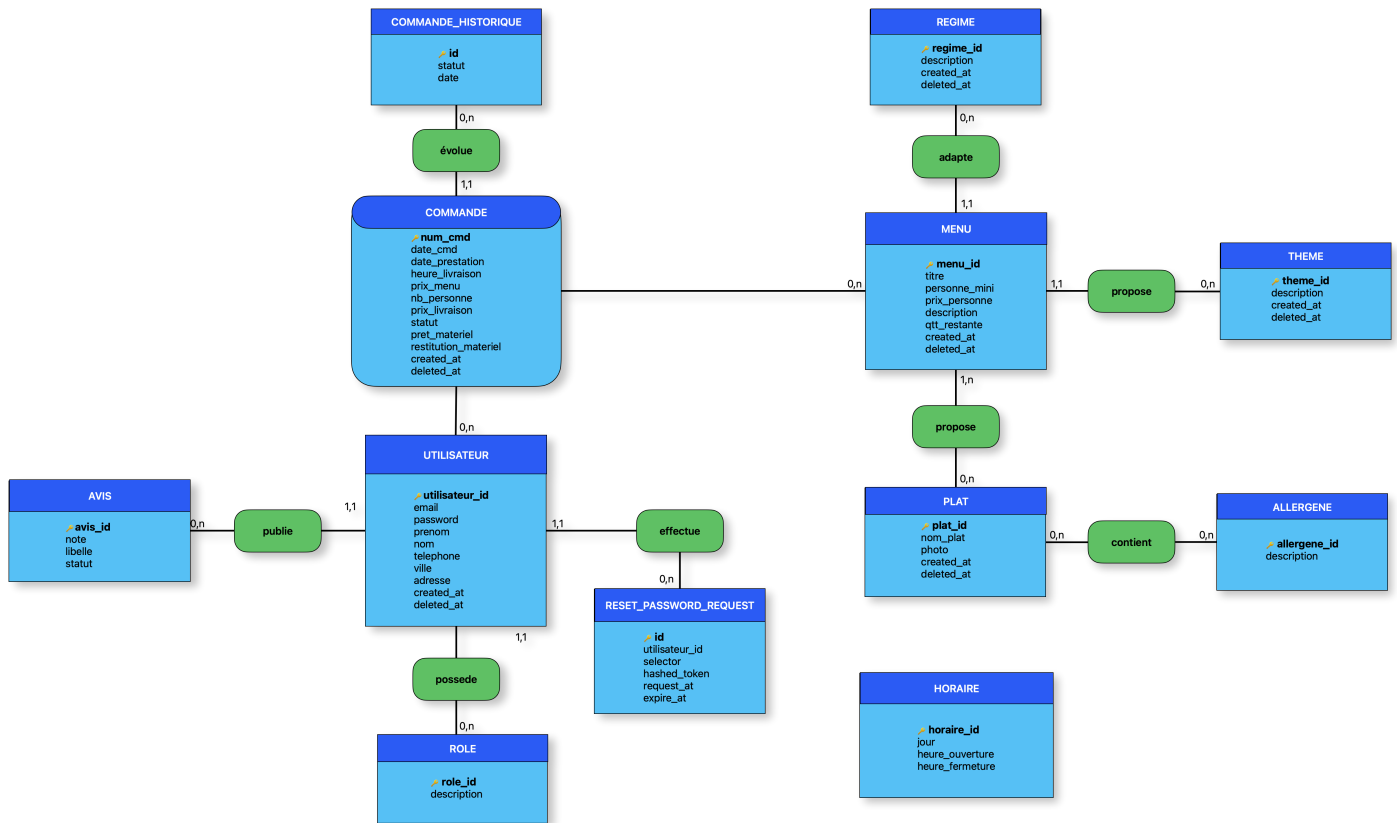


```
# Service Mysql
mysql:
  image: mysql:8.0
  ports:
    - "3307:3306"
```

Dans ce genre de cas, il faut faire **attention** à ne pas avoir de base de données MySQL installées en local, sinon **un conflit de ports** peut se produire. MySQL Workbench risque de se connecter à une base de données locales.

• Diagramme Méthode Merise

MCD



MLD

❑ PK (Primary Key)

❑ FK (Foreign Key)

PLAT: **plat_id**, nom_plat, photo

REGIME: **regime_id**, description, created_at, deleted_at

THEME: **theme_id**, description, created_at, deleted_at

MENU: **menu_id**, titre, personne_mini, prix_personne, description, qtt_restante, created_at, deleted_at,

theme_id, **plat_id**, **regime_id**

MENU_PLAT: **menu_id**, **plat_id**

ALLERGENE: **allergene_id**, description

PLAT_ALLERGENE: **plat_id**, **allergene_id**

ROLE: **role_id**, description

AVIS: **avis_id**, libelle, statut, **utilisateur_id**

UTILISATEUR: **utilisateur_id**, email, password, prenom, nom, telephone, ville, adresse, created_at, deleted_at, **role_id**

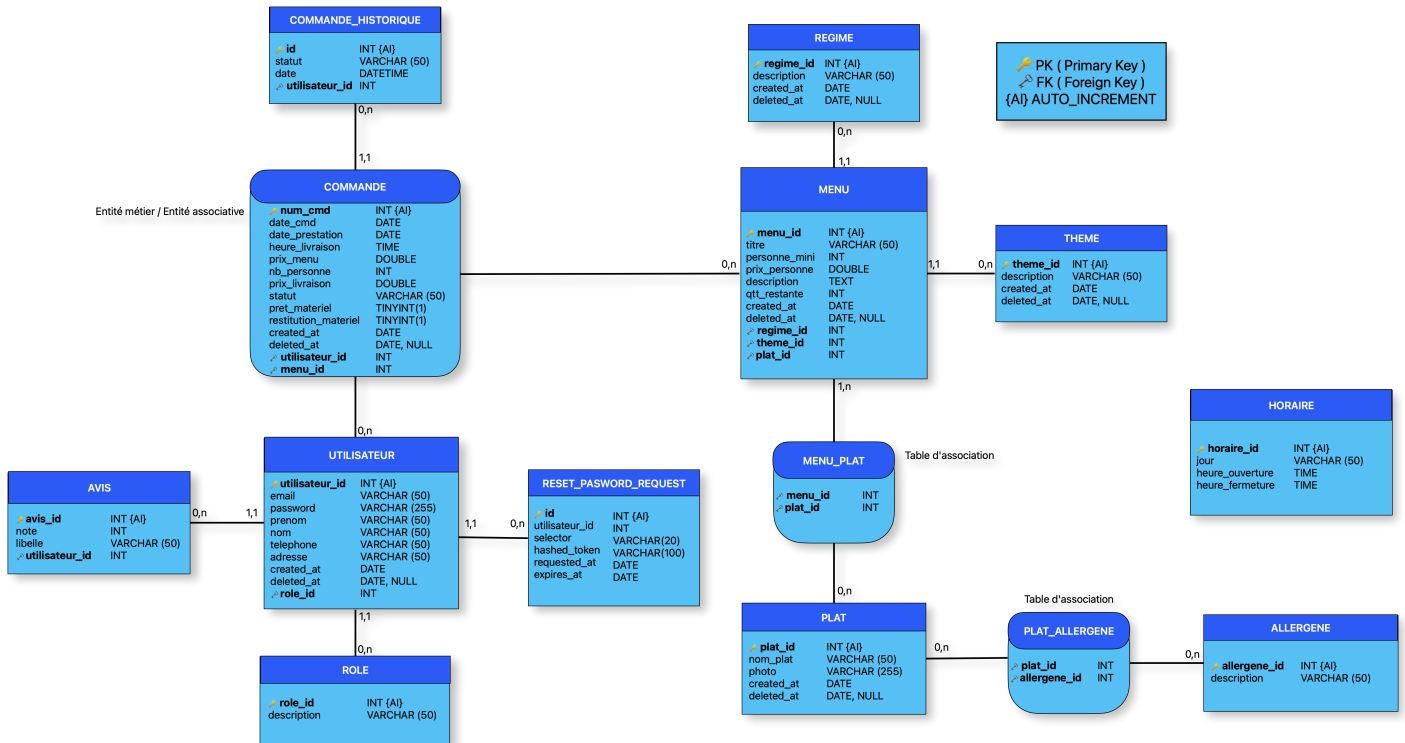
RESET_PASSWORD_REQUEST: **id**, selector, hashed_password, requested_at, expires_at, **utilisateur_id**

COMMANDE: **num_cmd**, date_cmd, date_prestation, heure_livraison, prix_menu, nb_personne, prix_livraison, statut, pret_materiel, restitution_materiel, created_at, deleted_at, **utilisateur_id**, **menu_id**

COMMANDE_HISTORIQUE: **id**, statut, date, **commande_id**

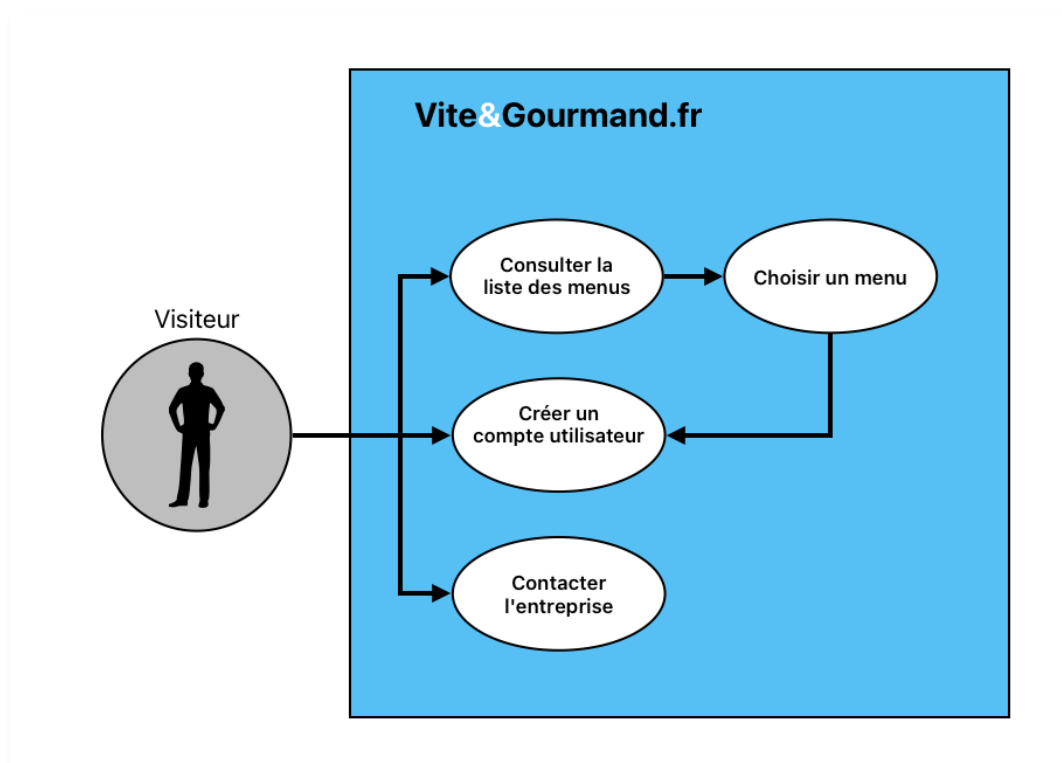
HORAIRE: **horaire_id**, jour, heureouverture, heurefermeture

MPD

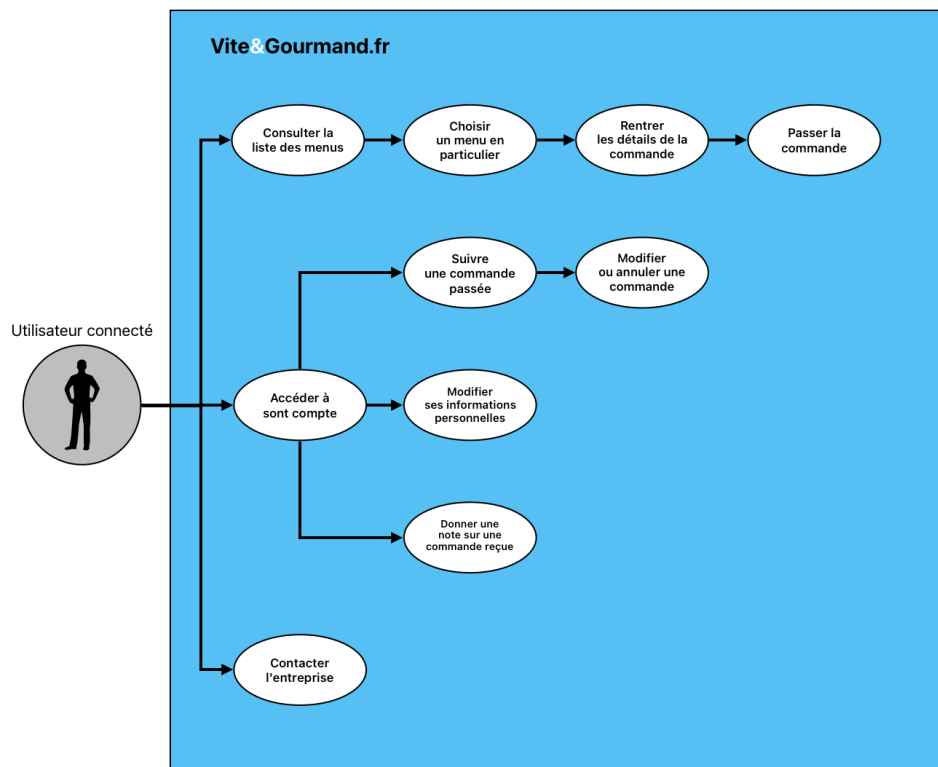


• Diagramme d'utilisation UML

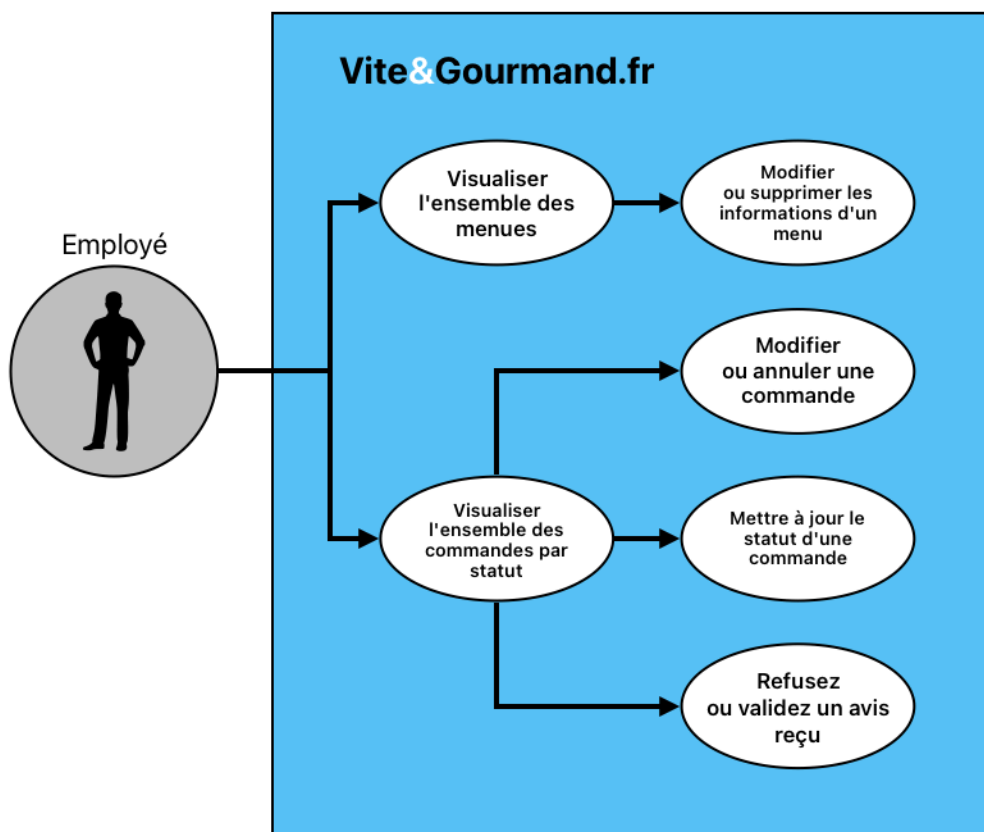
Use case visiteur



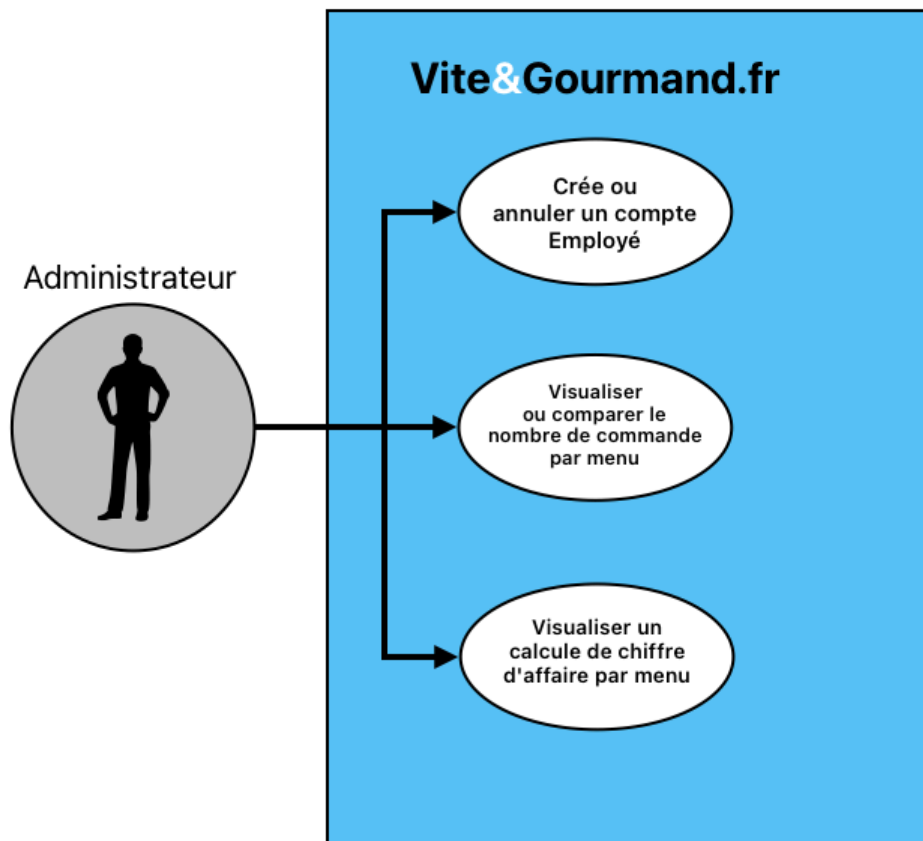
Use case utilisateur



Use case employé



Use case administrateur



• Diagramme de séquence d'une commande

